Lista de Exercícios: Tabelas Hash em JavaScript

Prof. Me. Alexandre Gomes

Resolver usando `Map` e `Set`. Cada um tem uma versão ingênua que funciona, mas é lenta. O objetivo é chegar à versão com hash, que faz uma única passada pelos dados.

Como trabalhar:

- Rodem no Node ou no console do navegador e confiram com os exemplos de cada exercício.
- Antes de programar, respondam: qual dado eu preciso consultar várias vezes, e em que estrutura ele fica para a consulta ser O(1)?

Exercício 1: Detectar duplicados

Escreva `temDuplicado(arr)`, que devolve `true` se algum valor aparece mais de uma vez no array e `false` caso contrário.

Entrada	Saída esperada
`[1, 2, 3, 1]`	`true`
`[1, 2, 3]`	`false`

Dica: o que um `Set` diz sobre um valor que você já viu?

Pense: a versão com dois laços aninhados custa O(n^2). Qual é o custo da sua versão?

Exercício 2: Frequência de palavras

Escreva `frequenciaPalavras(texto)`, que devolve um `Map` com cada palavra e quantas vezes ela aparece. Ignore maiúsculas e pontuação.

Entrada	Saída esperada
`"O gato viu o rato e o rato viu o gato"`	`o` = 4, `gato` = 2, `viu` = 2, `rato` = 2, `e` = 1

Dica: `mapa.get(p) || 0` devolve a contagem atual, ou zero se a palavra ainda não apareceu.

Exercício 3: Two Sum

Escreva `twoSum(nums, alvo)`, que devolve os índices `[i, j]` de dois números do array que somam `alvo`, ou `null` se não existirem. Faça em uma única passada.

Entrada	Saída esperada
`twoSum([2, 7, 11, 15], 9)`	`[0, 1]`
`twoSum([3, 2, 4], 6)`	`[1, 2]`
`twoSum([1, 2], 10)`	`null`

Dica: para cada número, qual complemento (`alvo - número`) você precisaria ter visto antes? Onde guardar o que já foi visto?

Exercício 4: Primeiro caractere não repetido

Escreva `primeiroUnico(s)`, que devolve o primeiro caractere da string que aparece uma única vez, ou `null` se não houver.

Entrada	Saída esperada
`"abacabad"`	`"c"`
`"aabb"`	`null`

Dica: use duas passadas: uma para contar e outra para procurar.

Exercício 5: Agrupar anagramas

Escreva `agruparAnagramas(palavras)`, que devolve um array de grupos, cada grupo com as palavras que são anagramas entre si.

Entrada	Saída esperada
`["amor", "roma", "ramo", "gato", "toga", "casa"]`	`[["amor", "roma", "ramo"], ["gato", "toga"], ["casa"]]`

Dica: o que "amor", "roma" e "ramo" têm em comum depois de terem as letras ordenadas? Isso pode servir de chave?

Exercício 6: Fibonacci com memoização

Escreva `fibMemo(n)`, que calcula o n-ésimo número de Fibonacci guardando em um `Map` os resultados já calculados.

Entrada	Saída esperada
`fibMemo(10)`	`55`
`fibMemo(50)`	`12586269025`

Compare, com `console.time`, o tempo de `fibMemo(40)` com o de uma versão recursiva sem `Map`.

Desafio

Na `HashTable` com encadeamento feita em aula, acrescente dois métodos: `keys()`, que devolve um array com todas as chaves guardadas, e `maiorBalde()`, que devolve o tamanho da maior lista. Insira 1000 chaves (`"aluno0"` até `"aluno999"`) e mostre o resultado de `maiorBalde()`.